

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа 3

Выполнил:

Лошкарёв Д. В.

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Основная цель этого задания заключалась в том, чтобы превратить ручное тестирование API в автоматизированный процесс. Мне нужно было не просто проверить, отвечают ли эндпоинты, а выстроить из них полноценный логический сценарий, который имитирует реальный путь пользователя. Важно было доказать, что все части системы - от регистрации до работы с базой данных - корректно взаимодействуют друг с другом в единой цепочке.

Ход работы

Я начал с настройки окружения (Environment) в Postman. Это позволило мне вынести адрес сервера в отдельную переменную baseUrl. Такой подход очень удобен: если я решу запустить тесты на удаленном сервере вместо локального докера, мне достаточно будет изменить значение в одном месте, и все запросы в коллекции обновятся автоматически.

Главной фишкой работы стала автоматизация передачи данных между запросами. Для этого я использовал скрипты на языке JavaScript во вкладке Tests. Например, после запроса на авторизацию я написал скрипт, который забирает JWT-токен из ответа сервера и сохраняет его в память Postman. Теперь во всех последующих запросах этот токен подставляется в заголовки сам. Вот как выглядел этот небольшой кусок кода:

```
// Сохранение токена после логина const jsonData = pm.response.json();
pm.test("Токен успешно получен", function () {
  pm.expect(jsonData.token).to.be.a('string'); pm.environment.set("jwt_token",
  jsonData.token); });
```

Сам сценарий я выстроил максимально логично. Сначала идет регистрация нового пользователя, затем вход в систему под этими данными. После получения прав доступа я выполняю поиск рецепта по названию. Чтобы тест был по-настоящему автоматическим, я добавил скрипт, который берет ID первого найденного рецепта из списка и запоминает его для следующего шага.

```
// Сохранение ID найденного рецепта const searchData =  
pm.response.json();  
pm.test("Рецепты найдены", function () {  
  pm.expect(searchData.items.length).to.be.above(0);  
  pm.environment.set("current_recipe_id", searchData.items[0].id); });
```

Завершается вся цепочка постановкой лайка выбранному рецепту. Здесь проверяется сразу всё: и работа базы данных, и правильность обработки JWT-токена, и корректность связей в таблицах. После того как я прописал тесты для каждого шага, я запустил всю коллекцию через Collection Runner. Это специальный инструмент в Postman, который прогоняет все запросы один за другим и выводит отчет.

The screenshot shows the 'Run results' for a collection named 'Recipe API Tests'. The interface includes a table with summary statistics and a detailed list of test iterations.

Source	Environment	Iterations	Duration	All tests	Avg. Resp. Time
Runner	Recipe Project	1	842ms	5	40 ms

Below the table, it indicates 'All Tests Passed (5) Failed (0) Skipped (0)' and provides a 'View Summary' link.

Iteration 1

- POST /auth/register**
http://localhost:3000/auth/register
Status: 201 • 102 ms • 536 B • 1
Result: PASS Успешная регистрация
- POST /auth/login**
http://localhost:3000/auth/login
Status: 200 • 75 ms • 531 B • 1
Result: PASS Токен получен
- GET /recipes/search?query=Паста**
http://localhost:3000/recipes/search?query=Паста
Status: 200 • 8 ms • 722 B • 1
Result: PASS Рецепты найдены
- GET /recipes/{{first_recipe_id}}**
http://localhost:3000/recipes/1
Status: 200 • 12 ms • 700 B • 1
Result: PASS Детали рецепта получены
- POST /recipes/{{first_recipe_id}}/like**
http://localhost:3000/recipes/1/like
Status: 200 • 3 ms • 254 B • 1
Result: PASS Лайк поставлен

Такой прогон подтвердил, что бэкенд стабилен и готов к нагрузке. Если бы где-то в коде закралась ошибка (например, перестал бы работать

поиск или сломалась авторизация), Runner бы сразу подсветил проблемный шаг красным цветом.

Вывод

В ходе выполнения домашнего задания я научился писать автоматизированные сценарии тестирования, которые экономят кучу времени при разработке.

Использование переменных окружения и скриптов внутри Postman позволяет проверять не просто отдельные функции, а целостность всей бизнес-логики приложения. Это критически важный навык, так как ручная проверка каждой кнопки при каждом изменении кода — это путь к ошибкам, а автоматика гарантирует, что проект всегда остается в рабочем состоянии.